Witness-CL: Delayed Corroboration for Executable Memory

Stage 1 Registered Report

Samuel Mausberg

September 2026

Abstract

Witness-CL extracts parameterized relations from an agent's successful SQL and makes them executable only after a later direct solution corroborates unchanged code under a changed binding. Typed compilation and exact receipts expose whether a new outer computation uses the relation on fresh rows. A finite-witness counterexample separates observed execution dependence from transfer guarantees. This Stage 1 manuscript reports a prospective protocol and a separately frozen exploratory diagnostic: two streams, three policies and 240 completed episodes. Full history, official-source ACE and delayed memory answered every question correctly, including 16/16 final probes per arm. Delayed memory used 40.9% fewer total tokens than full history but 115.4% more than ACE. One model-generated chain progressed from a source SUM through changed-binding corroboration to a correct COMPOSE with MEAN; emptying its relation made the saved program wrong. The sole frozen solver rerun after relation removal remained correct. Thus the diagnostic observes executable reuse without an accuracy advantage or a cost advantage over ACE. The original 32-stream pilot remains incomplete at 161/17,664 episodes. Confirmatory superiority, two-point noninferiority, drift robustness and native stateful gain remain unestablished.

Claims at this revision

Established in the two-stream diagnostic

1. All three arms answered all 80 assigned questions correctly, including 16/16 final probes each.
2. Delayed memory used 40.9% fewer total tokens than full history and 115.4% more than ACE.
3. One changed-binding chain reached correct new-outer composition; the sole solver rerun after relation removal remained correct.

Not established

1. Confirmatory accuracy superiority with 20% fewer tokens against both controls.
2. Population-level transfer or two-point old-task noninferiority.
3. Drift robustness or native-benchmark stateful gain.

1 The contribution to test

Persistent memory changes an agent's future inputs, but a changed input does not by itself establish learning. A model can copy an old scalar, repeat a working query, or solve a question again from documentation. A program library can accumulate entries that are never executed. Our proposed contribution is a specific connection: a relation extracted from one successful computation becomes available only after later evidence corroborates its parameterization, and its contribution to a new computation is evaluated separately.

Consider a warehouse agent that sums a quantity for one customer region. A later question asks for that sum in another region. If the unchanged relation reconstructs both direct solutions with different bindings and outputs, it can enter the executable library. A still later question requests a maximum or variance over its rows on a fresh database. The last computation is the transfer event of interest. Storing the first scalar is inadequate. Storing exact SQL as text might be sufficient, which is why a strong control must be allowed to do so.

The research target is a conjunction. The mechanism must cause useful transfer, the complete agent must outperform competent controls under a declared resource constraint, and protected earlier competence must remain adequate. A handful of convincing examples cannot compensate for a failed population comparison. Lower tokens at lower accuracy cannot establish efficiency, and an inconclusive old-panel interval cannot establish retention.

Earlier Witness-CL controllers, neural isolation experiments, feature learners and kernel measurements are preserved in the companion technical report [5]. This paper concerns one frozen-backbone SQL memory mechanism, its bounded exploratory evidence, and the independent confirmation still needed to assess it.

2 Setting and legal evidence

An independent stream contains sequential scalar questions over changing SQLite databases. Physical names are opaque. A learner-visible catalog describes units, NULL handling, joins and current-profile conventions.

Every ordinary episode has fresh rows. Schema and question text can remain unchanged when data change, so the interface supplies a fresh episode nonce and explicitly identifies prior numeric results as historical observations.

The learner receives its question, schema, permitted persistent memory and current tool results. No gold scalar, reference SQL, evaluator recipe, latent family identifier, convention bit vector, seed or future schedule enters its prompt. After an ordinary answer, the learner receives a correctness bit. Python oracle values and independent SQLite reference queries belong to the evaluator and are retained for audit rather than used to construct memory.

2.1 A common current-execution interface

Every arm begins with the same charged SELECT of the current catalog. This control prevents the executable arm from receiving a free drift detector. It also makes semantic evidence available equally to full history and evolving text. The observation consumes one of eight SELECT attempts per episode.

The solver has at most five model actions, each permitting at most 4096 generated tokens. The current response policy gives every arm a brief planning field and an action field; ACE additionally keeps its official citation field. Planning text asks the model to check relevant public catalog conventions and is fully charged. A `QUERY` contains read-only SQL, named scalar parameters and an `answer` flag. When that flag is true, the host accepts a complete finite numeric scalar from the query just executed and submits that exact value. There is no separately supplied number that can replace the current receipt. Invalid JSON consumes a model action; failed SQL consumes a SELECT attempt. Exploratory queries and learning checks share the ordinary query allowance.

Executable arms additionally have `USE` for selected relations and `COMPOSE` for structured outer computations. `PROGRAM` is an alias for the same compiler. The resulting scalar uses the same current-answer path. A rejected operation leaves ordinary direct querying available within the remaining allowance. It does not restore spent resources or supply a reference answer for free.

Current execution closes the channel in which the model submits an old scalar without querying. It does not certify intent: a constant SELECT can still return a copied number, and an executed query can implement the wrong interpretation. The common interface is therefore a control for freshness, rather than evidence for representation learning.

2.2 What changes during deployment

Let M_t be persistent memory after episode t, and let θ denote fixed pretrained weights. Learning updates external state:

$$M_{t+1} = U(M_t, H_{t+1}), \qquad \pi_t = \pi_\theta(\cdot \mid M_t). \tag{1}$$

The complete policy includes retrieval, prompt construction, decoding, compiler, executor, fallback and budgets. No online gradient update or offline retraining occurs. The model generates its own source SQL, while the host's supported extraction grammar supplies an explicit inductive bias.

Correctness on question x is $A(\pi, x) \in \{0, 1\}$. We record model input and output tokens, SELECT attempts, model calls, latency and retained bytes separately. The absence of weight training does not make acquisition or reflection free. Failed answers remain outcomes, and lower cost alone does not establish an improvement.

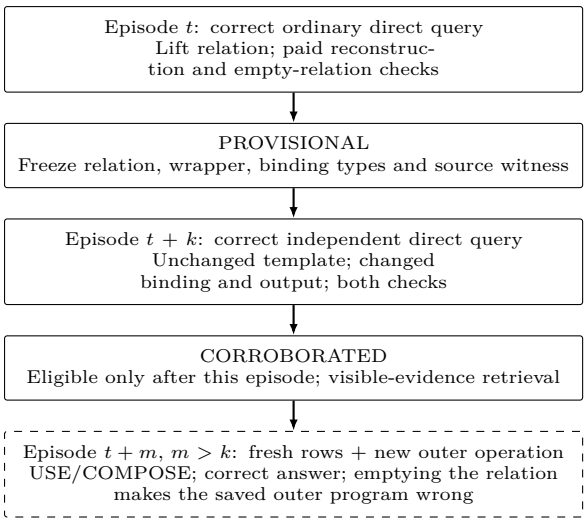

Figure 1: Implemented lifecycle and execution-dependence criterion. One model-generated chain in the two-stream diagnostic reaches the last box. Emptying a relation in a saved program and removing it before a new solver attempt are distinct interventions.

3 Delayed corroboration

The lifecycle separates three milestones. A *provisional* relation has a correct source answer and paid reconstruction and dependence checks. A *corroborated* relation has an additional later direct witness with a changed binding and output. A *transfer event* occurs only afterward, when the relation executes in a new outer computation on fresh rows and answers correctly. The first two are implementation states; the third is an empirical classification that requires the execution audit.

3.1 Extract an executed computation

After a correct direct answer, a deterministic parser lifts supported row-level computation from the terminal

query into an immutable relation, preserving columns and lineage. Supported row-selection literals become typed binding holes. A reconstruction wrapper retains the source outer operation. The host does not receive a gold relation or evaluator-authored query: the source is the model's own executed computation.

Lifting is a declared host transformation, not another model proposal call or unrestricted abstraction discovery. Unsupported SQL acquires no relation. Structurally different but equivalent SQL need not match. A relation retains its predicates and cannot expose rows filtered out by the source. These limits constrain what the method can learn and should be visible in its admission coverage.

Let q be the successful source query on database d, with observed scalar $y = q(d)$. For lifted relation $f(d; b)$ and wrapper w, acquisition spends one SELECT on reconstruction:

$$w(f(d; b)) = y. \tag{2}$$

It then spends another SELECT after emptying the relation while preserving its columns:

$$w(\varnothing_f) \neq w(f(d; b)). \tag{3}$$

Errors, truncation or incompatible results reject the candidate. NULL can be a valid changed result when the original answer was finite. An unchanged aggregate rejects admission even when the relation might help on another fixture. Every attempted check is retained and charged.

These checks run only if two SELECT attempts remain within the original episode allowance. They cannot revise the submitted score or become new source witnesses. A promising query with exhausted allowance receives no free learning phase. Acquisition consequently competes with exploration and ordinary problem solving.

3.2 Freeze what the evidence supports

A provisional record contains immutable relation and wrapper code, column information, typed binding holes, compatibility scope and source receipt identities. Template matching ignores the concrete supported row-binding values but preserves their types and the code being checked. Parameters belonging only to the outer wrapper remain fixed. A later observation cannot rewrite the relation and then be counted as evidence for the original object.

Compatibility uses the visible physical schema and a canonical complete catalog observation, preserving duplicate catalog rows. It contains no hidden task ID and excludes ordinary row values. Thus a changed aggregate does not by itself invalidate a relation, while a changed documented convention makes its old scope ineligible. This contract assumes accurate current documentation; it cannot detect an undocumented change of meaning.

3.3 Require a later direct witness

On a later ordinary episode, the agent first answers through a direct query. The provisional candidate is not exposed as executable memory for obtaining that answer. After correct feedback, the host lifts the new source and compares its normalized template and scope with provisional entries. The binding and observed scalar must both differ from the initial witness. Episode identifiers and indices enforce the temporal order.

If there is a match, the host executes the original relation and original wrapper under the new typed binding. It compares the result with the new own direct answer and repeats the empty-relation check. Both SELECTs are charged. Unsupported SQL, mismatching output, unchanged output or insufficient budget leaves the entry unpromoted. A successful second witness makes the entry eligible only on subsequent episodes; same-episode reuse cannot supply the later transfer event.

Two varying witnesses eliminate the simplest repeated-constant evidence but do not identify the intended relation. A misleading proxy can match both. A useful relation may also never recur in a compatible form. The experiment measures these possibilities instead of treating the registry's state as a certificate of useful generalization.

3.4 Execute a different operation

Retrieval ranks compatible eligible entries using learner-visible question information. The model sees selected SQL, columns and lineage and decides whether to compose or query directly. Hidden family labels and reference programs do not determine selection.

Composition uses a structured expression language whose leaves name selected relations, declared columns, typed values and supported operations. Physical table access stays inside the learned relations. The outer program cannot name an arbitrary table and independently recompute the source while leaving the learned relation unused. The generated request still passes the ordinary SQL authority boundary. These restrictions prevent that bypass but do not prove that the exposed rows answer the new question.

A qualifying transfer uses fresh row data and an outer operation different from source reconstruction and corroboration. Changed-binding reuse is measured separately. Direct SQL copied from visible memory and guard-only observations can affect whole-agent accuracy, but they do not count as execution through the learned-relation route. Reporting both prevents a zero execution count from being mistaken for proof that memory had no influence.

4 What the checks establish

The implementation combines structural restrictions with finite observations. It does not turn successful observations into a semantic certificate. The following narrow results determine how to interpret admission, transfer and retention.

4.1 Finite witnesses leave a transfer gap

Proposition 2 (Finite dependence does not establish transfer)**.** *For every finite witness set $W \subset \mathbb{N}$, there exists a candidate that matches reference answer 1 on every witness, changes its output when its relation is removed on every witness, and gives a wrong answer on a fresh context while still changing under that intervention.*

Proof. Let $f_W(x, 1) = 1$ if $x \in W$ and 2 otherwise; let $f_W(x, 0) = 0$. The second argument represents relation availability. Every witness gives the correct answer and intervention difference. Choose $x^* = 1 + \sum_{x \in W} x$; it is outside W, so the candidate's nonintervened answer is $2 \neq 1$ and the intervention still gives 0. The Lean construction uses a finite list of natural numbers and proves the required fresh element and all equalities. □

The result is an abstract counterexample, not a claim that the model generated this function or that arbitrary SQL has been verified. It establishes a narrow logical boundary: any transfer guarantee needs additional restrictions on the candidate class, observations or evaluation distribution. The proof is not a sample-complexity lower bound for the implemented learner.

4.2 The enforced SQL boundary

Typed parameters transport integer, finite real, text and NULL values separately from SQL syntax. Inner and outer namespaces remain separate. Supported source and composition forms have explicit limits. The executor rejects writes, external databases and disallowed functions. Current-answer submission resolves a result from the present episode, and post-answer learning cannot change that score. Panel evaluations use checkpoint clones and commit no state.

Implementation checks cover binding types, action parsing, access restrictions, reconstruction, dependency and snapshot restoration. Replay executes compiled requests in SQLite and checks the recorded outcomes. These checks do not supply a formal refinement proof of the Python compiler or SQLite's implementation.

4.3 Closed continuations

Let $F(z, u)$ be a full system transition, including memory, routing, input construction, tools, randomness and remaining resources. If old and new transitions agree on a protected set P, and their common successor remains in P, induction from an initial state in P over an input tape gives equal output traces and final states. The preserved Lean contract is `closed_continuations_equal`. Equality of one current action is insufficient without the full successor and closure premises.

The SQL system supplies no verified closed-state invariant for its evolving policy. A rejected operation changes the prompt and remaining budget for fallback. Immutable stored code therefore does not preserve the old policy's complete computation. This contract explains the need to evaluate retention under the whole interface; it does not prove that the statistical endpoint passes.

4.4 The accounting bridge

For a finite protected population of C contexts, let $T(p)$ sum a policy's integer returns in $[0, B_r]$. For update j, let $F_j = \mathbf{1}\{T(p_j) + \tau_j < T(p_{j-1})\}$ with $\tau_j \geq 0$. The deterministic Lean bridge establishes

$$T(p_0) \leq T(p_n) + \sum_j \tau_j + CB_r \sum_j F_j, \tag{4}$$

and the corresponding bound from each historical checkpoint. Actual population violations occur in the identity; an observed audit rejection is not the same quantity. Individually tolerated losses can accumulate against an original reference.

The campaign evaluates a single after-versus-before old-task comparison at the specified checkpoints. It does not grant every promotion a lifetime no-forgetting certificate. Lean proves the accounting identity, not independent sampling, model usage reports, distributional stability or useful statistical power. Equal population means also need not preserve every individual context.

Resource feasibility. Let L be acquisition cost, c_h a competent direct-query cost, and c_m the checked-use cost. Reusing a candidate n times saves SELECTs only if

$$L + nc_m < nc_h. \tag{5}$$

With a separately charged guard, $c_m = g + e$. If $c_h = 1$, $g \geq 1$ and $e \geq 1$, the inequality is impossible for $L \geq 0$ and $n > 0$. When $c_h > c_m$, the break-even requirement is $n > L/(c_h - c_m)$. This accounting observation is not a general lower bound on all memory systems: a different validator or task can have different costs. It does rule out SELECT savings for that path against a one-query baseline. The campaign's shared catalog query is charged to every arm and must enter each actual cost comparison. Token and latency gains remain distinct, measurable hypotheses.

5 Experimental protocol

This Stage 1 protocol separates the current pilot from subsequent sizing, confirmation, causal diagnostics and native benchmark contact. The planned decision point is a complete 32-stream pilot. The runtime amendment below adds a separate exploratory run while leaving confirmation and the planned causal controls deferred. A source/configuration freeze precedes each new block and binds prompts, updates, generator, model and backend identities, decoding, retrieval, budgets, split identifiers and assigned schedules. Pilot and qualification observations are excluded from confirmation.

5.1 Backbone and runtime

The first candidate used Qwen3-Coder-30B-A3B-Instruct; both complete cold qualifications failed. The current candidate uses the dense Qwen3.6-27B model and the shared planning interface. These changes were made together during development, so qualification does not estimate their separate effects. Each candidate has a fixed official publisher revision and fresh qualification seeds. Each weight, configuration and tokenizer file is checked against its publisher LFS SHA-256 or Git blob identity. A clean pinned llama.cpp checkout converts those inputs to BF16, recording the output hash, conversion environment and backend shared libraries. Conversion uses CPU and offline local inputs. GH200 serving uses a 65,536-token context within the native limit, one slot and no YaRN extension.

Decoding is nonthinking with temperature .7, top-p .8, top-k 20, minimum probability zero, repetition penalty 1.0, a full-context repetition window, and zero presence and frequency penalties. Identical fields enter preflight and generation receipts. All arms share the backbone and executor; method-specific prompts remain part of the evaluated policy. A shared seed is not a guarantee of bitwise GPU determinism, so actual outputs are preserved. The first candidate used repetition penalty 1.05. The dense candidate's neutral penalties were fixed before its first output: this backend also penalizes prompt tokens, including SQL identifiers that must be copied exactly.

The full-pilot protocol has no short campaign wall deadline; the subsequent exploratory run has the explicit cutoff below. A pre-launch check compares free GPU memory with the model, cache estimate and reserve. Generic transport calls are setup evidence and their costs are separate from study costs. Failed generation attempts retain their recorded usage, including an explicit unknown marker when measurement is unavailable.

5.2 The complete schedule

Each main stream begins with eight ordinary learning questions, followed by a 64-instance frozen old panel balanced over those eight families. Sixteen further ordinary questions supply changed bindings and new operations. The same old panel then measures the acquired policy again, followed by 32 final questions. Each probe uses an isolated clone of the corresponding checkpoint; panel answers and feedback never become subsequent memory.

Final-panel accuracy is the future endpoint. Initial and intermediate scores remain descriptive learning-curve outcomes. The old panel measures before-to-after change on a fixed protected distribution. Reusing its hidden instances reduces measurement noise without providing extra training evidence. Confirmation has no warm-accuracy early stop: every assigned outcome remains in its complete schedule.

The pilot includes the four already frozen development streams and 28 additional prospectively assigned streams with the identical learner and schedule. The first block remains intact; no observed outcome selects, replaces or excludes a stream from the 32. Qualification episodes are not pilot replications. The full-pilot protocol requires all assigned streams even if no qualifying mechanism event occurs; the runtime amendment does not redefine that denominator.

Before sizing, the report gives the full mechanism census, including zero counts; each arm's final-panel accuracy distribution and proportion of perfect streams; and the empirical distribution and standard deviation of paired old-panel changes. A ceiling in control accuracy limits the feasible superiority effect. A constant observed retention change must be reported explicitly rather than interpreted as proof that the population variance is zero.

Qualification's eight final questions use only `MEAN`. The pilot's full 32-question final panel also includes `MAX`, `VARIANCE` and `MEAN_SQUARE`, so perfect qualification does not establish a ceiling on the pilot panel. Pilot ceiling and variance estimates apply to the pilot distribution. The held, unlaunched out-of-distribution confirmation template changes ordinary operations, final operations, variance bindings and joint convention combinations. It requires matching calibration. The default proposal is fresh independent streams from the pilot generator; the target population must align with calibration, or receive additional calibration, before power is frozen. No final confirmation protocol is set at this revision.

Within each stream, arms receive the same questions, snapshots, catalog and budgets but learn from their own actions and feedback. Their histories are not made artificially equal after divergent decisions. Across streams, environment and solver randomization are separately seeded. Inferential analysis reduces the observations to one contrast vector per independent stream; panel rows do not become independent replications.

5.3 Runtime-budget amendment

A subsequent resource-budget amendment limited further execution to two hours. The original pilot was paused at an episode boundary on 9 September 2026 at 22:50 UTC, with 161 of 17,664 assigned episode records saved and no pending or orphan model calls. These counts record progress, not a completed pilot result. All 32 assigned streams and their original schedules remain in the record. Their original server-process binding requires a separate amendment before they can resume under the restarted server.

A separate exploratory diagnostic assigned fresh seeds 101302 and 101303 to all three arms with the same qualified dense model, learner, interface, decoding and development generator. Each arm–stream cell contains 24 ordinary questions, four old probes before, the same four after, and eight final probes: 240 episode records in total. The zero-based old indices are $0, 3, 4, 6$ from the original 64-probe panel; final indices are $0, 3, 9, 12, 18, 23, 29, 30$ from the original 32-question panel. Selection is frozen before this new run and supplies two final questions per outer operation. It does not supply a full retention panel.

An initial diagnostic freeze for seeds 101300 and 101301 remains unused: the original server process disappeared for an undetermined reason, and launch checks stopped before any model call. The replacement freeze followed a restart verified against the same model bytes, backend, configuration and launch command, with no smoke generation.

The diagnostic set a new-call cutoff of 10 September 2026 at 00:37 UTC, with reporting targeted for 00:43 UTC; all 240 episodes finished at 00:12:51 UTC. Its separate freeze and receipts preserve the 240-record assignment, actual completion, all measured costs and any missing cells. It supplies descriptive accuracy and mechanism observations only. It cannot complete the 32-stream pilot, calibrate its retention variance, establish noninferiority, or support a confirmatory superiority claim.

5.4 Principal controls

Full history retains complete legal ordinary interactions, including working SQL, errors, documentation, answers and feedback. It can query directly on every later question. Its context ceiling is explicit; silent truncation would change the comparator. Budget exhaustion is reported as an outcome rather than repaired after inspecting confirmation.

ACE uses official-pinned generator, reflector and curator prompts, incremental playbook additions, stable bullet IDs and helpful/harmful counters. The generator's final-answer envelope is adapted to the common SQL actions. Reflection uses the no-ground-truth path with ordinary observations and Boolean feedback. The playbook may retain SQL and exact conventions. Generator, reflection and curation calls are charged. The optional bullet analyzer is disabled. The pinned merger supports ADD operations; this differs from CL-Bench's modified ACE merger. The active playbook cap is 65,536 bytes and the curator's target is 16,384 tokens. This is a declared implementation of that path, not a claim to cover every ACE variant.

Endpoint	Required lower bound
Future vs history	$\mathbb{E}[F_D - F_H] > 0$
Future vs ACE	$\mathbb{E}[F_D - F_A] > 0$
Tokens vs history	$\mathbb{E}[.8T_H - T_D] > 0$
Tokens vs ACE	$\mathbb{E}[.8T_A - T_D] > 0$
Old-task retention	$\mathbb{E}[O_{D,\text{after}} - O_{D,\text{before}}] > -.02$

Table 1: Five co-primary comparisons, each at one-sided level .01. Old-task comparisons with other arms are secondary.

Delayed corroboration combines exact own evidence with the lifecycle in Figure 1. It pays reconstruction and dependence checks within the same eight-SELECT allowance. Active model-context bytes, stored relations, provenance registry and peak serialized memory are reported separately. A small active prompt does not imply bounded lifetime audit storage.

5.5 Five co-primary endpoints

Let D, H, A denote delayed memory, history and ACE. Let F, T, O denote final accuracy, total model tokens and old-panel accuracy. Table 1 defines the five paired stream contrasts. Every endpoint must pass before the full positive headline is stated.

Tokens include every scheduled phase, acquisition, rejected checks, reflection and failed generation attempts with measured usage. Paired cost differences test a ratio of aggregate expected costs. The mean of per-stream ratios would answer a different question. SELECTs, latency and retained bytes remain additional measured costs, not substitutes for the token endpoint.

The proposed analysis uses one-sided paired-stream Student-t bounds at level .01. Bonferroni controls familywise error at .05 under valid individual tests. The current analysis implementation uses the larger of observed paired standard deviation and .10 for accuracy, or .10 times mean control tokens for a cost contrast. The retention floor is under review pending measured pilot variance; it is not fixed by this Stage 1 manuscript. The two-point retention margin is retained. The final variance treatment must be specified before independent confirmation. A bound crossing its threshold fails that endpoint even when its point estimate is favorable. Missing required panels or unknown cost measurements prevent the complete positive claim.

5.6 Prospective sizing after the pilot

The pilot measures contrast variability and the room for a future accuracy gain before confirmation size is frozen. Reference planning alternatives are a five-point accuracy gain, token ratio .70 against each control, and zero old-task change. The observed final-panel ceiling must support the assumed accuracy effect. A true token ratio exactly .80 cannot give high power to prove a ratio below .80.

For a chosen planning standard deviation $\widetilde{\sigma}_j$ and slack d_j, allocate Type-II error .04 to each endpoint. Five individual powers of at least .96 imply joint power at least .80 by the union bound, without assuming endpoint independence. Token contrasts are normalized by the respective pilot control's mean cost. The normal approximation is

$$N \geq \max_j \left\lceil (z_{.99} + z_{.96})^2 \widetilde{\sigma}_j^2 / d_j^2 \right\rceil , \tag{6}$$

At standard deviation .10, a five-point accuracy effect needs about 67 streams, while two-point retention needs about 416. Noncentral-t sizing gives approximately 70 and 419. These are sensitivity calculations: neither a .10 retention floor nor a 419-stream schedule is a fixed commitment. Sizing each endpoint for only .80 power would not target .80 for their conjunction.

After the complete pilot report, prospective sizing will specify the variance treatment, feasible planning effects, joint-power simulation and fixed N. It cannot increase N in response to inspected confirmation outcomes. An assumed variance floor is not a distribution-free upper confidence bound; normal and simulation-based power remain conditional on their assumptions. The sizing receipt must preserve measured variances, ceiling diagnostics, simulation settings and the resulting schedule. Confirmation and the planned causal controls remain deferred until this pilot review is complete; the runtime amendment supplies a separate exploratory report.

6 Controls for alternative explanations

The principal comparison evaluates complete independently learning policies. Separate matched-history interventions address narrower causal questions, preserving their additional costs and denominators. Matched-evidence, admission-timing, drift and nonreuse comparisons remain deferred. The separately frozen single relation-deletion rerun is reported with the exploratory diagnostic.

6.1 Identical derived SQL as text

The view-text arm shares source extraction, paid checks, corroboration and stored definitions, but exposes them for ordinary `QUERY` use rather than executable composition. It receives the same derived SQL, columns and lineage. This asks whether the execution interface adds value beyond a good textual record of the computation. If text matches accuracy at lower total cost, the executable-interface advantage is rejected.

Exposure must be matched as well as storage. Text cannot be denied working SQL, nor can the program arm receive hidden semantic annotations. Differences in compiler instructions and action envelopes remain declared parts of the interface rather than being described as pure compression.

6.2 The same extra direct queries

Acquisition and corroboration yield extra observations. A control receiving only the ordinary source query therefore cannot distinguish representation reuse from additional evidence. The matched-evidence diagnostic gives a control the same extra permitted direct-query observations at corresponding checkpoints, with full costs and exact query/result text.

Matched-history forks are distinct from the principal independently learning arms. A fork begins from a recorded public-history checkpoint under a frozen assignment rule. It adds no gold answer, evaluator reference or future label. The report identifies which observations were shared and how their queries were charged. A principal accuracy gain is not attributed to representation reuse if this extra-evidence control accounts for it.

6.3 Immediate admission

An immediate-admission arm makes a checked source relation eligible on later episodes without requiring the changed-binding witness. It shares source extraction, initial checks and execution machinery. The contrast separates the effect of delay from the effect of executable memory. Delay can prevent weak reuse but lose valuable early opportunities; both belong in its measured cost.

Different future decisions can generate different later memories, so the independent-arm contrast measures the whole policy effect. A matched-history fork supplies the narrower comparison when the same source object must be held fixed.

6.4 Drift and nonreuse

Stable, drift and nonreuse conditions distinguish actual recurrence from superficial similarity. Drift changes documented conventions while keeping enough question and schema structure to tempt stale reuse. Every arm sees the same charged current catalog. The library's scope check should withhold old entries under changed documentation, but the complete agent's adaptation is an empirical outcome.

The old panel stays on its original protected distribution. It measures retained competence while current drift questions measure adaptation. Combining them would hide the difference between preserving obsolete answers and forgetting useful old behavior. The catalog key does not detect undocumented semantic change, and that limitation remains part of the interpretation.

The proposed diagnostics use 64 fresh paired stable/drift streams and 32 nonreuse streams. Nonreuse changes vocabulary and conventions to reduce applicability of accumulated relations. It exposes acquisition overhead without frequent recurrence. Diagnostic rows do not increase the principal confirmatory sample size, and a favorable drift result does not replace a failed primary endpoint.

6.5 Causal trace requirements

A qualifying chain links three distinct receipts in order: admission, changed-binding corroboration, then later execution. Audit checks immutable template identity, types, changed binding, observed output variation, compatibility, fresh row hashes and a new outer operation. Operationally, the auditor requires a new aggregate kind, join, keyed grouping or multiple grouping stages; it does not decide unrestricted semantic novelty. The final action must actually execute the relation and answer correctly. A guard observation or direct query containing remembered SQL does not satisfy that route requirement.

Two interventions answer different questions. Holding the saved outer program and database fixed while emptying the relation tests instance-level execution dependence. Removing the relation from memory and rerunning the solver tests whether the complete policy compensates through direct solving. The first can change the answer while the second remains correct.

The strongest policy-level trace claim requires a correct original answer and a wrong answer from an actual solver rerun after relation removal. A fixed-program empty-relation intervention alone does not establish it. Errors are reported explicitly. All qualifying chains are counted; any displayed examples follow a frozen deterministic ordering rather than author selection. Examples illustrate the mechanism but do not establish its prevalence without the full denominator of eligible opportunities.

7 Native benchmark contact

CL-Bench evaluates stateful-versus-stateless gain in native stateful domains [1]. Planned native contact uses DatabaseExploration, the official database and default questions, and five official orderings. Task construction and the upstream evaluator remain pinned and unchanged. The custom generator is a mechanism fixture, not a relabeled native benchmark.

The native action envelope is ordinary `QUERY`/`ANSWER`. Checks must occur before `ANSWER`, and registry updates commit only after successful ordinary feedback. The adapter cannot append post-answer queries disallowed by the environment or import custom-fixture reference answers. Unsupported source computations remain ordinary queries without executable promotion.

Stateful and stateless policies share backbone, limits and scoring. Stateless runs reset learned memory between instances. Their paired score difference is reported with total costs and actual relation executions. Five orderings of one database are not five independent databases; they support a bounded native-domain comparison rather than general benchmark superiority.

ACE uses the same legal observations and charges all generator, reflector and curator calls. Response-schema adaptations are documented. Interface parity tests demonstrate protocol compatibility. Scripted fixtures and generic transport checks do not supply benchmark scores. Native stateful gain is reported only after end-to-end model runs on the real domain.

8 Stage 1 evidence and measurement status

This Stage 1 registered-report manuscript has no complete, audit-bound confirmatory analysis. Primary outcomes are unmeasured; pending, failed-integrity and scripted inputs supply no effect estimate. Completed descriptive results appear separately below.

8.1 Mechanism observations

Complete study	Qualifying executions
fast-dense-v2-restart	1

Complete mechanism census. Counts require chronological admission, changed-binding corroboration, fresh rows, new outer operation, and wrong outcomes after fixed-program empty-relation and computed-measure interventions. These do not establish an agent rerun after deleting a registry entry.

All qualifying events, lineage, intervention outcomes and eligible counts are retained in the bound status artifact. A deterministic first-five selection is reported there for every completed study; studies with zero events remain visible.

8.2 Descriptive controls and benchmark contact

Matched-history fork costs include each complete donor prefix; overlapping checkpoint costs are not added as physical model usage. Native permutations reuse fixed databases and do not count as independent custom streams. A negative drift or stateful-gain result remains

Primary comparison	Estimate	One-sided lower bound	Decision
Future accuracy: delayed − history	*Unmeasured*	*Unmeasured*	Not evaluated
Future accuracy: delayed − ACE	*Unmeasured*	*Unmeasured*	Not evaluated
Total token ratio: delayed / history	*Unmeasured*	*Unmeasured*	Not evaluated
Total token ratio: delayed / ACE	*Unmeasured*	*Unmeasured*	Not evaluated
Old accuracy: after − before	*Unmeasured*	*Unmeasured*	Not evaluated

Table 2: Proposed complete-stream primary report; final sizing follows the 32-stream pilot. Bounds use one-sided $\alpha = .01$. Accuracy bounds are percentage points; token bounds apply to $0.8C - D$ in tokens, not to the displayed aggregate ratio. Thresholds are zero except −2 pp for retention.

Complete study	Evidence class	Schedule	Outcome status
coder-v1 / qualification-drift	qualification	2 streams; drift	Qualification fails; 39/64 correct
coder-v1 / qualification-reuse	qualification	2 streams; reuse	Qualification fails; 33/64 correct
dense-v2 / qualification-drift	qualification	2 streams; drift	Qualification passes; 64/64 correct
dense-v2 / qualification-reuse	qualification	2 streams; reuse	Qualification passes; 64/64 correct
fast-dense-v2-restart	Descriptive diagnostic	2 streams; reuse	Descriptive; no primary claim
fast-dense-v2-restart-deletion	agent_deletion	All 1 frozen eligible events	0/1 answers flip to wrong

Table 3: Complete audited studies, including negative results. Full per-arm counts and costs appear in the accompanying status artifact; distinct studies are never pooled to enlarge the confirmation denominator.

part of the report rather than becoming a reason to replace its schedule.

8.3 Provenance and reporting boundary

The machine-readable status and its readable companion record every discovered study, input hashes, source and schedule bindings, completed record counts, measured costs and integrity failures. The publisher makes no model calls and accepts primary estimates only after recomputing the frozen analysis from complete audited panels. Missing measurements are never replaced with zeros, development estimates or test-double outputs. Earlier GH200 development does not contribute observations to these campaign tables. Generic transport smoke calls are recorded separately in the status artifact, including failed setup attempts with unknown usage. They are not study observations.

8.4 Exploratory diagnostic under the amendment

All 240 assigned episode records completed and passed independent replay. The six arm–stream cells each scored 24/24 ordinary questions, 4/4 old probes before, 4/4 old probes after, and 8/8 final questions. Thus every arm was at the observed ceiling, with no paired accuracy gain. Each arm's two retention differences were zero (sample standard deviation zero, one variance degree of freedom), with no discordance among its eight paired old probes.

Arm	Final	Calls	Total tokens
Full history	16/16	85	2,278,924
ACE	16/16	180	625,254
Delayed	16/16	88	1,347,094

Table 4: Complete two-stream exploratory diagnostic. Each arm answered all 80 scheduled questions correctly. Tokens include all ordinary, old and final episodes, acquisition, rejected actions and ACE updates; setup and the separate deletion rerun are excluded.

Neither the two streams nor the reduced four-probe old panels establish a two-point noninferiority result or calibrate the full pilot's retention variance. The original pilot remains at 161/17,664 records; these exploratory observations do not fill its missing cells.

The complete diagnostic used 353 model calls and 4,251,272 measured tokens, with no unknown usage. Delayed memory's aggregate token ratios were 0.5911 against full history and 2.1545 against ACE (Table 4). Its 40.9% saving against history therefore coexists with 115.4% greater cost than ACE. This is a negative result for the proposed competitive conjunction in the measured diagnostic, without a population-level inference.

The delayed-arm census records 36 own-source admissions and 12 later corroborations, yielding 12 ever-eligible relations. Twelve relations were retrieved, three executed and four evicted; one chain qualifies. Two old-panel executions retain the source outer operation and do not

qualify. One final execution in seed 101303 qualifies: ordinary episode 3 admitted a North-region net-order-value relation after SUM returned 2987.75; episode 11 corroborated the unchanged relation for South after SUM returned 2643.00. Final probe 3, from the checkpoint after 24 ordinary episodes, used COMPOSE to compute MEAN over the North binding on fresh rows, returning 322.67857142857144 correctly. The first COMPOSE omitted its binding and was rejected; the second supplied it. Both model calls and the rejected action are included in the reported costs.

The three distinct database hashes and source/corroboration/execution receipts are retained at frozen schedule ordinals 129, 165 and 219. Emptying the relation changed the saved program's result to NULL; replacing its computed measure with zero gave 0, and replacing it with NULL gave NULL. All three intervention outputs were wrong. This is one qualifying chain among 80 delayed-arm episodes, including 16 final probes, rather than a handful of independent transfer examples.

The complete eligible set contained one actual relation-deletion rerun. With the registry relation removed and the original SQL evidence retained, the solver issued a direct QUERY and returned the same correct value, 322.67857142857144. The independent audit replayed this single frozen case: zero policy-level wrong-answer flips in one actual rerun. It cost one model call and 18,777 measured tokens, with no unknown usage. The diagnostic plus this separate rerun therefore used 354 calls and 4,270,049 tokens. The result establishes recovery in this case, not absence of causal effects in general.

9 Related work

Voyager accumulates executable skills with feedback, and Agent Workflow Memory induces reusable routines during agent interaction [9, 10]. A program library or retry loop is therefore not a novelty claim. The narrower question is whether delayed evidence about unchanged parameterized code produces correct future composition that repays its complete acquisition cost.

Dynamic Cheatsheet retains reusable text and code, while ACE evolves context incrementally [7, 12]. Evo-Memory's ReMem connects reasoning, action and memory refinement [11]. These methods motivate controls that retain exact conventions and working SQL. Our declared ACE path is not a reimplementation of ReMem or every evolving-text method.

AgentCL uses compositional streams to diagnose memory and continual learning [6]. Its distinction between related-task gains and regressions motivates our separate binding, new-operation, nonreuse and old-panel measurements. CL-Bench supplies complementary native contact. Controlled fixtures support interventions; native tasks test an existing interaction contract. Neither form of evidence replaces the other.

SQLSolver proves supported query equivalences; VeriEQL checks bounded database equivalence and can produce counterexamples [2, 3]. Neither is integrated here. Even source query equivalence would leave the intended semantics of the source and useful composition with a new operation unresolved. This work is not a general SQL verification system.

High-confidence policy improvement and confidence sequences provide established statistical tools under sampling assumptions [4, 8]. We claim no new concentration theorem. The experimental obligations are frozen comparisons, independent streams, accounted evidence and a clear distinction between checkpoint noninferiority and lifetime pointwise preservation.

10 Interpretation and limitations

The exploratory result separates observed reuse from competitive advantage. One checked relation supports a new outer operation, yet all three policies already answer every diagnostic question correctly and ACE uses the fewest tokens. These observations do not establish the proposed superiority-and-cost conjunction. The sole solver rerun after relation removal reconstructed a correct direct query, so this case also does not demonstrate policy-level necessity. These observations do not estimate the unmeasured full pilot or the held out-of-distribution population.

The fixture deliberately supplies related computations. Its supported grammar, documented conventions and eight initial families limit external validity. Unconditional row generation can produce empty or degenerate results that block corroboration. Such episodes remain in the sample. Removing them after inspecting confirmation would change the target distribution. Native contact adds evidence but does not eliminate the generator's question selection or the backbone's pretraining knowledge.

Exact template matching can miss equivalent sources. Extraction can reject useful SQL; retrieval can omit a useful relation; compatible code can still have wrong row semantics. Documentation equality is only as reliable as documentation. Active-memory caps can evict useful evidence, while retained audit provenance consumes storage even when hidden from the model's context.

Precision also has a computational cost. Larger old panels reduce within-stream noise but do not eliminate common-history correlations. More panel questions cannot replace independent streams when between-stream variation is large. Observed variance and the limits of the sizing model must accompany the final bounds.

No result is created by changing a deadline rule. The runner checkpoints and resumes the same schedule. Unknown usage remains an accounting gap until resolved from authentic records. Missing panels block the con-

junction. A failed confirmation can motivate a new development cycle, with new streams and a new freeze, but cannot be repaired by selecting favorable old episodes.

11 Reproduction and artifact contract

The release separates source/configuration freezes, raw model and SQL receipts, memory checkpoints and derived reports. The freeze binds prompt construction, updates, extraction, compiler, evaluator, model weights, decoding and backend. Resume checks those identities before continuing. Model output never grants authority to change the evaluator or SQL executor.

Replay reconstructs frozen databases, executes recorded SQL and compares result rows, submitted values and memory transitions. It checks chronology, source eligibility, scope and later binding changes. Cost reduction sums every model attempt, including reflection and learning, while keeping missing measurements explicit. Independent oracle checks compare Python calculations and SQLite references before model conclusions are drawn.

An offline reporting comparison initially differed in lineage traversal order. The corrected report sorts lineage node and child lists while preserving their contents and multiplicity, and excludes replay timing from equality. It independently reproduces all 240 records and the same event census. The frozen sources, original report failure and raw observations remain unchanged.

The principal interfaces are the episode executor, immutable registry snapshots, the campaign client and the checkpointed runner. The runtime preparation tool owns publisher verification, conversion provenance and setup receipts. These boundaries let a reader trace an aggregate or displayed example back to actual observations rather than a regenerated summary alone.

The result section uses audited campaign artifacts. Complete exploratory observations and missing confirmatory endpoints remain distinct. The finite-witness proof is retained in `formal/WitnessCL/Intervention.lean`; the closed-continuation and accounting statements retain their original conditional scope. Formal builds, compiler tests and model performance remain distinct evidence classes.

Authorship. This manuscript and implementation used AI assistance.

References

[1] Parth Asawa, Christopher M. Glaze, Gabriel Orlanski, Ramya Ramakrishnan, Benji Xu, Asim Biswal, Vincent Sunn Chen, Frederic Sala, Matei Zaharia, and Joseph E. Gonzalez. Continual learning bench: Evaluating frontier AI systems in real-world stateful environments. arXiv:2606.05661, 2026. URL `https://arxiv.org/abs/2606.05661`.

[2] Haoran Ding, Zhaoguo Wang, Yicun Yang, Dexin Zhang, Zhenglin Xu, Haibo Chen, Ruzica Piskac, and Jinyang Li. Proving query equivalence using linear integer arithmetic. *Proceedings of the ACM on Management of Data*, 1 (4):227:1–227:26, 2023. doi: 10.1145/3626768. URL `https://www.cs.yale.edu/homes/piskac/papers/2024SIGMOD.pdf`.

[3] Yang He, Pinhan Zhao, Xinyu Wang, and Yuepeng Wang. VeriEQL: Bounded equivalence verification for complex SQL queries with integrity constraints. *Proceedings of the ACM on Programming Languages*, 8 (OOPSLA1):132:1–132:29, 2024. doi: 10.1145/3649849. URL `https://arxiv.org/abs/2403.03193`.

[4] Steven R. Howard, Aaditya Ramdas, Jon McAuliffe, and Jasjeet Sekhon. Time-uniform, nonparametric, nonasymptotic confidence sequences. *The Annals of Statistics*, 49(2):1055–1080, 2021. doi: 10.1214/20-AOS1991. URL `https://arxiv.org/abs/1810.08240`.

[5] Samuel Mausberg. Witness-CL technical report: Constructive contracts and historical evidence, 2026. Companion technical report supplied with this manuscript. PDF.

[6] Yiheng Shu, Bernal Jiménez Gutiérrez, Saisri Padmaja Jonnalagedda, Yuguang Yao, Huan Sun, and Yu Su. AgentCL: Toward rigorous evaluation of continual learning in language agents. arXiv:2606.02461, 2026. URL `https://arxiv.org/abs/2606.02461`.

[7] Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. Dynamic cheatsheet: Test-time learning with adaptive memory. arXiv:2504.07952, 2025. URL `https://arxiv.org/abs/2504.07952`.

[8] Philip S. Thomas, Georgios Theocharous, and Mohammad Ghavamzadeh. High confidence policy improvement. In *Proceedings of ICML*, volume 37 of *PMLR*, pages 2380–2388, 2015. URL `https://proceedings.mlr.press/v37/thomas15.html`.

[9] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. arXiv:2305.16291, 2023. URL `https://arxiv.org/abs/2305.16291`.

[10] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. arXiv:2409.07429, 2024. URL `https://arxiv.org/abs/2409.07429`.

[11] Tianxin Wei, Noveen Sachdeva, Benjamin Coleman, Zhankui He, Yuanchen Bei, Xuying Ning, Mengting Ai, Yunzhe Li, Jingrui He, Ed H. Chi, Chi Wang, Shuo Chen, Fernando Pereira, Wang-Cheng Kang, and Derek Zhiyuan Cheng. Evo-memory: Benchmarking LLM agent test-time learning with self-evolving memory. arXiv:2511.20857, 2025. URL `https://arxiv.org/abs/2511.20857`. Version 2, revised May 2026.

[12] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. arXiv:2510.04618, 2025. URL `https://arxiv.org/abs/2510.04618`. Version 3, revised March 2026.